

privateSQL_tpch_query_transformation

January 7, 2026

1 Purpose

This notebook demonstrates that each TPC-H query transformed into a Tumult / openDP-friendly view plus a linearized query produces the same result as the original SQL query (no differential privacy applied). The workflow:

- create a decorrelated, deduped view that contains only the attributes needed for linear answering;
- express the original aggregate as a linear query over that view; and
- verify equality with the original query result.

Queries processed: Q1, Q4, Q13, Q16.

Verification - Each query runs the original SQL and the transformed (view + linear) SQL. - Results are compared using `pyspark.testing.assertDataFrameEqual` (or by row/rowcount check). Cells print either “The DataFrames are identical (schema and content match).” or a description of the difference. - Non-DP outputs (linearized results) are written to `./output/no_dp/q{N}_result_csv` and original results to `./output/orig/q{N}_result_csv`.

How to reproduce 1. Ensure raw TPC-H data is in `./raw` as `.tbl` files (pipe-delimited).
2. Run the data loading cells to populate Spark tables (cells that call `load_from_csv` and `create_empty_tables`).
3. Execute the original and transformed query cells for each query: 4. Check console output for the `assertDataFrameEqual` result and inspect the CSVs under `./output/orig` and `./output/no_dp`.

Notes - This file does not apply differential privacy; it confirms functional equivalence before DP instrumentation.

```
[4]: import os
os.environ["JAVA_HOME"] = "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/
↳Contents/Home"
os.environ["SPARK_HOME"] = "/Users/kinchintong/project/tumult-labs/spark-4.0.
↳1-bin-hadoop3"

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import findspark
findspark.init(os.environ["SPARK_HOME"])
from math import floor
```

```

from pyspark import SparkFiles
from pyspark.sql import SparkSession, types as T, functions as F
from tmlt.analytics.privacy_budget import PureDPBudget
from tmlt.analytics.protected_change import AddOneRow
from tmlt.analytics.query_builder import QueryBuilder
from tmlt.analytics.session import Session, ColumnType
from tmlt.analytics.keyset import KeySet
from functools import reduce
from tmlt.analytics import (
    AddOneRow,
    PureDPBudget,
    QueryBuilder,
    Session,
    AddRowsWithID,
)

from tmlt.analytics.truncation_strategy import TruncationStrategy

# Silence some Spark warnings
import warnings
warnings.simplefilter('ignore', UserWarning)
warnings.simplefilter('ignore', ResourceWarning)

findspark.init()

# Set up Spark
spark = (
    SparkSession.builder
        .appName("tpch-setup")
        .master("local[*]") # drop if connecting to a cluster
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.driver.memory", "16g")
        .config("spark.executor.memory", "16g")
        .config("spark.driver.maxResultSize", "2g")
        .getOrCreate()
)
print("Spark version:", spark.version)

spark.sparkContext.setLogLevel("ERROR")

spark.conf.set("spark.sql.session.timeZone", "UTC")

# CREATE EMPTY TABLES (schema only) ----
def create_empty_tables():
    for t in TABLES:
        empty = spark.createDataFrame([], SCHEMAS[t])
        # managed Parquet tables; change to .format("delta") if using Delta

```

```

        #empty.write.mode("overwrite").format("parquet").saveAsTable(t)
    print(f"Created empty schema in database `{DB}`.")

# LOAD FROM CSV / .tbl (pipe-delimited with trailing '/') ----
def load_from_csv(src_dir: str):
    for t in TABLES:
        schema = SCHEMAS[t]
        path = f"{src_dir}/{t}.tbl"    # change to .csv if needed
        #only if the path exists
        if not os.path.exists(path):
            print(f"Path does not exist: {path}")
            continue

        print(f"Loading {t} from {path}...")

        # create a temporary wide schema of string columns (named) to absorb
        # trailing '/'
        n_cols = len(schema.fields) + 1
        tmp_fields = [T.StructField(f"_c{i}", T.StringType(), True) for i in
        range(n_cols)]
        tmp_schema = T.StructType(tmp_fields)
        #print("tmp_schema:", tmp_schema)
        df_raw = (spark.read
                    .option("sep", "|")
                    .option("header", "false")
                    .option("mode", "PERMISSIVE")
                    .schema(tmp_schema) # read wide to absorb trailing '/'
                    .csv(path))

        #print(f"Raw schema for {t}:")
        #df_raw.printSchema()

        # keep only first N columns and cast to target schema
        cols = df_raw.columns[:len(schema.fields)]
        df = df_raw.select([F.col(c).alias(schema.fields[i].name) for i, c in
        enumerate(cols)])

        # cast columns to target types
        for f in schema.fields:
            if isinstance(f.dataType, T.DateType):
                df = df.withColumn(f.name, F.to_date(F.col(f.name),
                "yyyy-MM-dd"))

            elif isinstance(f.dataType, T.DecimalType):
                df = df.withColumn(f.name, F.col(f.name).cast(f.dataType))

            elif isinstance(f.dataType, T.IntegerType):
                df = df.withColumn(f.name, F.col(f.name).cast("int"))

            # strings need no cast
        df.write.mode("overwrite").format("parquet").saveAsTable(t)
    print(f"Loaded {t} from {path}")

```

Spark version: 4.0.1

```
[5]: DB = "tpch"                                # change as you like

# Define canonical TPC-H schemas ----
dec = lambda p, s: T.DecimalType(p, s)
SCHEMAS = {
    "region": T.StructType([
        T.StructField("r_regionkey", T.IntegerType(), False),
        T.StructField("r_name", T.StringType(), False),
        T.StructField("r_comment", T.StringType(), True),
    ]),
    "nation": T.StructType([
        T.StructField("n_nationkey", T.IntegerType(), False),
        T.StructField("n_name", T.StringType(), False),
        T.StructField("n_regionkey", T.IntegerType(), False),
        T.StructField("n_comment", T.StringType(), True),
    ]),
    "part": T.StructType([
        T.StructField("p_partkey", T.IntegerType(), False),
        T.StructField("p_name", T.StringType(), False),
        T.StructField("p_mfgr", T.StringType(), False),
        T.StructField("p_brand", T.StringType(), False),
        T.StructField("p_type", T.StringType(), False),
        T.StructField("p_size", T.IntegerType(), False),
        T.StructField("p_container", T.StringType(), False),
        T.StructField("p_retailprice", T.FloatType(), False),
        T.StructField("p_comment", T.StringType(), True),
    ]),
    "supplier": T.StructType([
        T.StructField("s_suppkey", T.IntegerType(), False),
        T.StructField("s_name", T.StringType(), False),
        T.StructField("s_address", T.StringType(), False),
        T.StructField("s_nationkey", T.IntegerType(), False),
        T.StructField("s_phone", T.StringType(), False),
        T.StructField("s_acctbal", T.FloatType(), False),
        T.StructField("s_comment", T.StringType(), True),
    ]),
    "partsupp": T.StructType([
        T.StructField("ps_partkey", T.IntegerType(), False),
        T.StructField("ps_suppkey", T.IntegerType(), False),
        T.StructField("ps_availqty", T.IntegerType(), False),
        T.StructField("ps_supplycost", T.FloatType(), False),
        T.StructField("ps_comment", T.StringType(), True),
    ]),
    "customer": T.StructType([
        T.StructField("c_custkey", T.IntegerType(), False),
```

```

        T.StructField("c_name",      T.StringType(),  False),
        T.StructField("c_address",   T.StringType(),  False),
        T.StructField("c_nationkey", T.IntegerType(), False),
        T.StructField("c_phone",     T.StringType(),  False),
        T.StructField("c_acctbal",   T.FloatType(),   False),
        T.StructField("c_mktsegment",T.StringType(),  True),
        T.StructField("c_comment",   T.StringType(),  True),
    ]),
    "orders": T.StructType([
        T.StructField("o_orderkey",    T.IntegerType(), False),
        T.StructField("o_custkey",     T.IntegerType(), False),
        T.StructField("o_orderstatus",  T.StringType(),  False),
        T.StructField("o_totalprice",   T.FloatType(),   False),
        T.StructField("o_orderdate",    T.DateType(),    False),
        T.StructField("o_orderpriority",T.StringType(),  False),
        T.StructField("o_clerk",        T.StringType(),  False),
        T.StructField("o_shippriority", T.IntegerType(), False),
        T.StructField("o_comment",      T.StringType(),  True),
    ]),
    "lineitem": T.StructType([
        T.StructField("l_orderkey",    T.IntegerType(), False),
        T.StructField("l_partkey",     T.IntegerType(), False),
        T.StructField("l_suppkey",     T.IntegerType(), False),
        T.StructField("l_linenum",     T.IntegerType(), False),
        T.StructField("l_quantity",    T.FloatType(),   False),
        T.StructField("l_extendedprice",T.FloatType(),   False),
        T.StructField("l_discount",    T.FloatType(),   False),
        T.StructField("l_tax",         T.FloatType(),   False),
        T.StructField("l_returnflag",  T.StringType(),  False),
        T.StructField("l_linestatus",  T.StringType(),  False),
        T.StructField("l_shipdate",    T.DateType(),    False),
        T.StructField("l_commitdate",  T.DateType(),    False),
        T.StructField("l_receiptdate", T.DateType(),    False),
        T.StructField("l_shipinstruct",T.StringType(),  False),
        T.StructField("l_shipmode",    T.StringType(),  False),
        T.StructField("l_comment",     T.StringType(),  True),
    ]),
}
TABLES = ␣
    ↪["region","nation","part","supplier","partsupp","customer","orders","lineitem"]
spark.sql(f"CREATE DATABASE IF NOT EXISTS {DB}")
spark.catalog.setCurrentDatabase(DB)

create_empty_tables()

```

Created empty schema in database `tpch`.

```
[6]: SRC_CSV_DIR = "./raw"      # where *.tbl live
load_from_csv(SRC_CSV_DIR)

# Quick smoke test:
spark.sql("SHOW TABLES").show(truncate=False)
# show record count for each table
for t in TABLES:
    print(f"Table {t} has {spark.sql(f'SELECT count(*) FROM {t}').
    ↪collect()[0][0]} records.")
```

```
Loading region from ./raw/region.tbl...
Loaded region from ./raw/region.tbl
Loading nation from ./raw/nation.tbl...
Loaded nation from ./raw/nation.tbl
Loading part from ./raw/part.tbl...
Loaded part from ./raw/part.tbl
Loading supplier from ./raw/supplier.tbl...
Loaded supplier from ./raw/supplier.tbl
Loading partsupp from ./raw/partsupp.tbl...
```

```
Loaded partsupp from ./raw/partsupp.tbl
Loading customer from ./raw/customer.tbl...
Loaded customer from ./raw/customer.tbl
Loading orders from ./raw/orders.tbl...
```

```
Loaded orders from ./raw/orders.tbl
Loading lineitem from ./raw/lineitem.tbl...
```

```
Loaded lineitem from ./raw/lineitem.tbl
```

```
+-----+-----+-----+
|namespace|tableName|isTemporary|
+-----+-----+-----+
|tpch     |customer |false      |
|tpch     |lineitem |false      |
|tpch     |nation   |false      |
|tpch     |orders   |false      |
|tpch     |part     |false      |
|tpch     |partsupp |false      |
|tpch     |region   |false      |
|tpch     |supplier |false      |
+-----+-----+-----+
```

```
Table region has 5 records.
Table nation has 25 records.
Table part has 200000 records.
```

Table supplier has 10000 records.
Table partsupp has 800000 records.
Table customer has 150000 records.
Table orders has 1500000 records.
Table lineitem has 6001215 records.

1.1 TPC-H Queries Evaluated

- **Q1 - Pricing Summary Report Query** This query reports the amount of business that was billed, shipped, and returned.
 - **Q4 - Order Priority Checking Query** This query determines how well the order priority system is working and gives an assessment of customer satisfaction.
 - **Q13 - Customer Distribution Query** This query seeks relationships between customers and the size of their orders.
 - **Q16 - Parts/Supplier Relationship Query** This query finds out how many suppliers can supply parts with given attributes. It might be used, for example, to determine whether there is a sufficient number of suppliers for heavily ordered parts.
-

1.1.1 Q1 — Pricing Summary Report

```
SELECT
    l_returnflag,
    l_linestatus,
    SUM(l_quantity) AS sum_qty,
    SUM(l_extendedprice) AS sum_base_price,
    SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
    SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
    AVG(l_quantity) AS avg_qty,
    AVG(l_extendedprice) AS avg_price,
    AVG(l_discount) AS avg_disc,
    COUNT(_) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '[DELTA]' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

```
[7]: # --- Original Q1 ---
DELTA = 90 # number of days before 1998-12-01
q1 = f"""
SELECT
    l_returnflag,
    l_linestatus,
    SUM(l_quantity) AS sum_qty,
    SUM(l_extendedprice) AS sum_base_price,
```

```

        SUM(CAST(l_extendedprice AS DOUBLE) * (1.0 - CAST(l_discount AS DOUBLE))) AS
        ↪sum_disc_price,
        SUM(CAST(l_extendedprice AS DOUBLE) * (1.0 - CAST(l_discount AS DOUBLE)) *
        ↪(1.0 + CAST(l_tax AS DOUBLE))) AS sum_charge,
        AVG(l_quantity) AS avg_qty,
        AVG(l_extendedprice) AS avg_price,
        AVG(l_discount) AS avg_disc,
        COUNT(*) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '{DELTA}' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
"""

# original query 1
print("Original Query 1 Results:")
q1_res = spark.sql(q1)
row_count = q1_res.count()

q1_res.show(truncate=False)

print(f"Row count: {row_count}")

q1_res.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/orig/q1_result_csv")

```

Original Query 1 Results:

```

+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+
|l_returnflag|l_linestatus|sum_qty    |sum_base_price    |sum_disc_price    |
|sum_charge  |avg_qty      |avg_price   |avg_price         |avg_disc          |
|count_order|
+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+
|A          |F           |991417.0    |1.4875047103800006E9|
|3.7734107E7|5.6586554400730194E10|5.375825713486996E10 |5.5909065222827805E10|2
5.522005853257337|38273.129734621805|0.049985295838451425|1478493      |
|N          |F           |1.4130821680541005E9 |1.4696492231943743E9|
|25.516471920522985|38284.46776084832 |0.05009342667421665 |38854        |
|N          |O           |7.447604E7  |1.1170172969773996E11|1.06118230307606E11

```

```

|1.1036704387249677E11|25.50222676958499
|38249.117988908256|0.049996586053780115|2920374    |
|R              |F              |3.7719753E7|5.656804138090004E10
|5.3741292684604004E10|5.5889619119831924E10|25.50579361269077
|38250.85462609968 |0.050009405830180625|1478870    |
+-----+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+-----+-----+
---+-----+

```

Row count: 4

1.1.2 Transform Q1 to separate view and linear query for Tumult (openDP)

```

[8]: # --- Decorrelated, deduped view for linear answering ---
q1_view = """
SELECT
    l_returnflag,
    l_linestatus,
    l_quantity,
    l_extendedprice,
    l_discount,
    l_tax,
    l_shipdate,
    CAST(l_extendedprice AS DOUBLE) * (1.0 - CAST(l_discount AS DOUBLE)) AS disc_price,
    CAST(l_extendedprice AS DOUBLE) * (1.0 - CAST(l_discount AS DOUBLE)) * (1.0 + CAST(l_tax AS DOUBLE)) AS charge
FROM lineitem;
"""

spark.sql(f"CREATE OR REPLACE TEMP VIEW V_q1 AS {q1_view}")

q1_linear = f"""
SELECT
    l_returnflag,
    l_linestatus,
    SUM(l_quantity) AS sum_qty,
    SUM(l_extendedprice) AS sum_base_price,
    SUM(disc_price) AS sum_disc_price,
    SUM(charge) AS sum_charge,
    AVG(l_quantity) AS avg_qty,
    AVG(l_extendedprice) AS avg_price,
    AVG(l_discount) AS avg_disc,
    COUNT(*) AS count_order
FROM V_q1
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '{DELTA}' DAY

```

```

GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
"""

print("Linearized Query 4 Results with view:")
q1_res_linear = spark.sql(q1_linear)
row_count = q1_res_linear.count()

q1_res_linear.show(truncate=False)

print(f"Row count: {row_count}")

q1_res_linear.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/no_dp/q1_result_csv")

print("Verify if transformed query result matches original query result:")
from pyspark.testing import assertDataFrameEqual
try:
    assertDataFrameEqual(q1_res, q1_res_linear)
    print("The DataFrames are identical (schema and content match).")
except AssertionError as e:
    print("The DataFrames are different.")
    print(e)

```

Linearized Query 4 Results with view:

```

+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+
|l_returnflag|l_linestatus|sum_qty      |sum_base_price      |sum_disc_price
|sum_charge      |avg_qty      |avg_price      |avg_disc
|count_order|
+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+
|A          |F          |991417.0      |1.4875047103800006E9
|3.7734107E7|5.6586554400730194E10|5.375825713486996E10 |5.5909065222827805E10|2
5.522005853257337|38273.129734621805|0.049985295838451425|1478493      |
|N          |F          |991417.0      |1.4875047103800006E9
|1.4130821680541005E9 |1.4696492231943743E9
|25.516471920522985|38284.46776084832 |0.05009342667421665 |38854        |
|N          |O          |7.447604E7   |1.1170172969773996E11|1.06118230307606E11
|1.1036704387249677E11|25.50222676958499
|38249.117988908256|0.049996586053780115|2920374        |
|R          |F          |3.7719753E7  |5.656804138090004E10

```

```
|5.3741292684604004E10|5.5889619119831924E10|25.50579361269077
|38250.85462609968 |0.050009405830180625|1478870      |
+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+
```

Row count: 4

Verify if transformed query result matches original query result:

[Stage 106:> (0 + 10) / 10]

The DataFrames are identical (schema and content match).

1.1.3 Q4 - Order Priority Checking Query

```
SELECT
  o_orderpriority,
  COUNT(*) AS order_count
FROM
  orders
WHERE
  o_orderdate >= DATE '[DATE]'
  AND o_orderdate < DATE '[DATE]' + INTERVAL '3' MONTH
  AND EXISTS (
    SELECT
      *
    FROM
      lineitem
    WHERE
      l_orderkey = o_orderkey
      AND l_commitdate < l_receiptdate
  )
GROUP BY
  o_orderpriority
ORDER BY
  o_orderpriority;
```

```
[9]: from datetime import datetime, timedelta

# Define the date parameter
start_date = "1993-07-01"

# --- Original Q4 ---
q4 = f"""
```

```

SELECT
    o_orderpriority,
    COUNT(*) AS order_count
FROM
    orders
WHERE
    o_orderdate >= DATE '{start_date}'
    AND o_orderdate < DATE '{start_date}' + INTERVAL '3' MONTH
    AND EXISTS (
        SELECT
            *
        FROM
            lineitem
        WHERE
            l_orderkey = o_orderkey
            AND l_commitdate < l_receiptdate
    )
GROUP BY
    o_orderpriority
ORDER BY
    o_orderpriority
"""

```

```

# original query 4
print("Original Query 4 Results:")
q4_res = spark.sql(q4)
row_count = q4_res.count()

q4_res.show(truncate=False)

print(f"Row count: {row_count}")

q4_res.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/orig/q4_result_csv")

```

Original Query 4 Results:

```

+-----+-----+
|o_orderpriority|order_count|
+-----+-----+
|1-URGENT       |10594      |
|2-HIGH         |10476      |
|3-MEDIUM      |10410      |
|4-NOT SPECIFIED|10556      |
|5-LOW          |10487      |

```

+-----+-----+

Row count: 5

1.1.4 Transform Q1 to separate view and linear query for Tumult (openDP)

A) Baseline Transformation (Q4 — Order Priority Checking) Purpose. Prepare a reusable order-level view with just the columns we'll need later.

The correlated **EXISTS** predicate (`l_commitdate < l_receiptdate`) is **not** handled in baseline—it's added in the *decorrelation* step.

Instantiate the template - TableList := { orders } - FilterAttrs := { o_orderdate }
- GroupAttrs := { o_orderpriority } - StructKey := { o_orderkey } - AttrList :=
GroupAttrs FilterAttrs StructKey

Baseline view

```
CREATE OR REPLACE TEMP VIEW V_q4_base AS
SELECT
  `AttrList`
FROM `TableList`;

CREATE OR REPLACE TEMP VIEW V_q4_base AS
SELECT
  o_orderkey,
  o_orderpriority,
  o_orderdate
FROM orders;
```

B) Decorrelation convert the inner query with exists to join

```
-- Replace EXISTS with a semi-join, then dedup orders:
CREATE OR REPLACE TEMP VIEW V_q4 AS
SELECT DISTINCT
  o.o_orderkey,
  o.o_orderpriority
FROM orders o
JOIN lineitem l
  ON l.l_orderkey = o.o_orderkey
WHERE l.l_commitdate < l.l_receiptdate;
```

C) moving filter operations

```
-- Replace EXISTS with a semi-join, then dedup orders:
CREATE OR REPLACE TEMP VIEW V_q4 AS
SELECT DISTINCT
  o.o_orderkey,
  o.o_orderpriority
FROM orders o
JOIN lineitem l
```

```

    ON l.l_orderkey = o.o_orderkey
WHERE l.l_commitdate < l.l_receiptdate;

-- Final online query (now correct):
SELECT o.o_orderpriority, COUNT(*) AS order_count
FROM V_q4 v
JOIN orders o ON o.o_orderkey = v.o_orderkey
WHERE o.o_orderdate >= DATE 'DATE'
    AND o.o_orderdate < DATE 'DATE' + INTERVAL '3' MONTH
GROUP BY o.o_orderpriority
ORDER BY o.o_orderpriority;

```

```

[10]: # --- Decorrelated, deduped view for linear answering ---
q4_view = """
SELECT DISTINCT
    o.o_orderkey,
    o.o_orderpriority
FROM orders o
JOIN lineitem l
    ON l.l_orderkey = o.o_orderkey
WHERE l.l_commitdate < l.l_receiptdate;
"""

spark.sql(f"CREATE OR REPLACE TEMP VIEW V_q4 AS {q4_view}")

start_date = "1993-07-01"
q4_linear = f"""
SELECT o.o_orderpriority, COUNT(*) AS order_count
FROM V_q4 v
JOIN orders o ON o.o_orderkey = v.o_orderkey
WHERE o.o_orderdate >= DATE '{start_date}'
    AND o.o_orderdate < DATE '{start_date}' + INTERVAL '3' MONTH
GROUP BY o.o_orderpriority
ORDER BY o.o_orderpriority;
"""

print("Linearized Query 4 Results with view:")
q4_res_linear = spark.sql(q4_linear)
row_count = q4_res_linear.count()

q4_res_linear.show(truncate=False)

print(f"Row count: {row_count}")

q4_res_linear.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/no_dp/q4_result_csv")

```

```

print("Verify if transformed query result matches original query result:")
from pyspark.testing import assertDataFrameEqual
try:
    assertDataFrameEqual(q4_res, q4_res_linear)
    print("The DataFrames are identical (schema and content match).")
except AssertionError as e:
    print("The DataFrames are different.")
    print(e)

```

Linearized Query 4 Results with view:

```

+-----+-----+
|o_orderpriority|order_count|
+-----+-----+
|1-URGENT       |10594      |
|2-HIGH         |10476      |
|3-MEDIUM      |10410      |
|4-NOT SPECIFIED|10556      |
|5-LOW          |10487      |
+-----+-----+

```

Row count: 5

Verify if transformed query result matches original query result:
The DataFrames are identical (schema and content match).

1.1.5 Q13 - Customer Distribution Query

```

SELECT
    c_count,
    COUNT(*) AS custdist
FROM (
    SELECT
        c_custkey,
        COUNT(o_orderkey) AS c_count
    FROM
        customer
    LEFT OUTER JOIN orders
        ON c_custkey = o_custkey
        AND o_comment NOT LIKE '%[WORD1]%[WORD2]%'
    GROUP BY
        c_custkey
) AS c_orders (c_custkey, c_count)
GROUP BY

```

```

        c_count
ORDER BY
    custdist DESC,
    c_count DESC;

```

```

[11]: #WORD1 is randomly selected from 4 possible values: special, pending, unusual,
      ↪express.
      #WORD2 is randomly selected from 4 possible values: packages, requests,
      ↪accounts, deposits.'

      #Values for substitution parameters:
      WORD1 = 'special'
      WORD2 = 'requests'

      # --- Original Q13 ---
      q13 = f"""
      SELECT
          c_count,
          COUNT(*) AS custdist
      FROM (
          SELECT
              c_custkey,
              COUNT(o_orderkey) AS c_count
          FROM
              customer
          LEFT OUTER JOIN orders
              ON c_custkey = o_custkey
              AND o_comment NOT LIKE '%{WORD1}%{WORD2}%'
          GROUP BY
              c_custkey
      ) AS c_orders (c_custkey, c_count)
      GROUP BY
          c_count
      ORDER BY
          custdist DESC,
          c_count DESC;
      """

      # original query 13
      print("Original Query 13 Results:")
      q13_res = spark.sql(q13)
      row_count = q13_res.count()
      q13_res.show(truncate=False)

      print(f"Row count: {row_count}")

      q13_res.coalesce(1).write \

```

```
.mode("overwrite") \
.option("header", "true") \
.csv("./output/orig/q13_result_csv")
```

Original Query 13 Results:

```
+-----+-----+
|c_count|custdist|
+-----+-----+
|0      |50005   |
|9      |6641    |
|10     |6532    |
|11     |6014    |
|8      |5937    |
|12     |5639    |
|13     |5024    |
|19     |4793    |
|7      |4687    |
|17     |4587    |
|18     |4529    |
|20     |4516    |
|15     |4505    |
|14     |4446    |
|16     |4273    |
|21     |4190    |
|22     |3623    |
|6      |3265    |
|23     |3225    |
|24     |2742    |
+-----+-----+
```

only showing top 20 rows
Row count: 42

A) Baseline Transformation (Q13 - Customer Distribution Query) Instantiate the template - TableList := { customer LEFT JOIN orders } - FilterAttrs := { c_custkey, o_orderkey, o_comment } - GroupAttrs := { c_custkey } - StructKey := { } - AttrList := GroupAttrs FilterAttrs StructKey

Baseline view

```
CREATE OR REPLACE TEMP VIEW V_q13_base AS
SELECT
  `AttrList`
FROM `TableList`;
```

B) Decorrelation convert the inner query with exists to join

```
CREATE OR REPLACE TEMP VIEW V_q13_base AS
SELECT
  c.c_custkey,
```

```

    o.o_orderkey,
    o.o_comment
FROM customer c
LEFT JOIN orders o
    ON o.o_custkey = c.c_custkey;

```

C) moving filter operations

```

CREATE OR REPLACE TEMP VIEW V_q13 AS
SELECT
    c_custkey AS custdist,
    COUNT(o_orderkey) AS c_count
FROM V_q13_base

WHERE o_comment NOT LIKE '%{WORD1}%{WORD2}%'
GROUP BY c_custkey;

SELECT
    c_count,
    COUNT(*) AS custdist
FROM V_q13
GROUP BY c_count
ORDER BY custdist DESC, c_count DESC;

```

```

[12]: WORD1 = 'special'
      WORD2 = 'requests'

# --- Decorrelated, deduped view for linear answering ---
q13_view = f"""
SELECT
    c.c_custkey,
    COUNT(o.o_orderkey) AS c_count
FROM customer c
LEFT JOIN orders o
    ON o.o_custkey = c.c_custkey
    AND o_comment NOT LIKE '%{WORD1}%{WORD2}%'
GROUP BY c.c_custkey;
"""

print(q13_view)
spark.sql(f"CREATE OR REPLACE TEMP VIEW V_q13 AS {q13_view}")

spark.sql(f"SELECT * FROM V_q13").show(truncate=False)

q13_linear = f"""
SELECT
    c_count,
    COUNT(*) AS custdist
FROM V_q13

```

```

GROUP BY c_count
ORDER BY custdist DESC, c_count DESC;
"""

print("Linearized Query 13 Results with view:")
q13_res_linear = spark.sql(q13_linear)
row_count = q13_res_linear.count()
print(f"Row count: {row_count}")
q13_res_linear.show(truncate=False)

q13_res_linear.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/no_dp/q13_result_csv")

from pyspark.testing import assertDataFrameEqual
try:
    assertDataFrameEqual(q13_res, q13_res_linear)
    print("The DataFrames are identical (schema and content match).")
except AssertionError as e:
    print("The DataFrames are different.")
    print(e)

```

```

SELECT
    c.c_custkey,
    COUNT(o.o_orderkey) AS c_count
FROM customer c
LEFT JOIN orders o
    ON o.o_custkey = c.c_custkey
    AND o_comment NOT LIKE '%special%requests%'
GROUP BY c.c_custkey;

```

```

+-----+-----+
|c_custkey|c_count|
+-----+-----+
|148      |21      |
|463      |18      |
|471      |0       |
|496      |21      |
|833      |13      |
|1088     |14      |
|1238     |8       |
|1342     |21      |
|1580     |5       |
|1591     |13      |
|1645     |14      |
|1829     |20      |

```

1959	0	
2122	17	
2142	0	
2366	9	
2659	19	
2866	20	
3175	22	
3749	7	

+-----+-----+

only showing top 20 rows

Linearized Query 13 Results with view:

Row count: 42

+-----+-----+

c_count	custdist	
---------	----------	--

+-----+-----+

0	50005	
9	6641	
10	6532	
11	6014	
8	5937	
12	5639	
13	5024	
19	4793	
7	4687	
17	4587	
18	4529	
20	4516	
15	4505	
14	4446	
16	4273	
21	4190	
22	3623	
6	3265	
23	3225	
24	2742	

+-----+-----+

only showing top 20 rows

The DataFrames are identical (schema and content match).

1.1.6 Original TPC-H Q16

“sql SELECT p_brand, p_type, p_size, COUNT(DISTINCT ps_suppkey) AS supplier_cnt
FROM part p JOIN partsupp ps ON p.p_partkey = ps.ps_partkey WHERE p_brand <>
{BRAND}' AND p_type NOT LIKE '{TYPE_PREFIX}%' AND p_size IN ({sizes_csv}) AND
ps_suppkey NOT IN (SELECT s_suppkey FROM supplier WHERE s_comment LIKE '%Cus-
tomer%Complaints%') GROUP BY p_brand, p_type, p_size ORDER BY supplier_cnt DESC,
p_brand, p_type, p_size

```
[13]: # Parameters you can tweak
BRAND = "Brand#12"
TYPE_PREFIX = "LARGE ANODIZED"
SIZES = [3, 6, 9, 12, 18, 24, 36, 48]
sizes_csv = ",".join(map(str, SIZES))

# --- Original Q16 ---
q16 = f"""
SELECT
    p_brand,
    p_type,
    p_size,
    COUNT(DISTINCT ps_suppkey) AS supplier_cnt
FROM part p
JOIN partsupp ps
    ON p.p_partkey = ps.ps_partkey
WHERE p_brand <> '{BRAND}'
    AND p_type NOT LIKE '{TYPE_PREFIX}%'
    AND p_size IN ({sizes_csv})
    AND ps_suppkey NOT IN (
        SELECT s_suppkey
        FROM supplier
        WHERE s_comment LIKE '%Customer%Complaints%'
    )
GROUP BY p_brand, p_type, p_size
ORDER BY supplier_cnt DESC, p_brand, p_type, p_size
"""

# original query 16
print("Original Query 16 Results:")
q16_res = spark.sql(q16)
row_count = q16_res.count()

q16_res.show(truncate=False)

print(f"Row count: {row_count}")

q16_res.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/orig/q16_result_csv")
```

Original Query 16 Results:

p_brand	p_type	p_size	supplier_cnt
Brand#25	PROMO BRUSHED STEEL	24	28
Brand#41	ECONOMY ANODIZED STEEL	48	28

Brand#41	MEDIUM BRUSHED TIN	3	28	
Brand#11	STANDARD BURNISHED BRASS	36	24	
Brand#14	PROMO PLATED COPPER	48	24	
Brand#15	MEDIUM ANODIZED NICKEL	3	24	
Brand#21	MEDIUM ANODIZED COPPER	3	24	
Brand#22	SMALL BRUSHED NICKEL	3	24	
Brand#22	STANDARD ANODIZED TIN	12	24	
Brand#25	MEDIUM BURNISHED COPPER	36	24	
Brand#31	PROMO POLISHED COPPER	36	24	
Brand#35	PROMO POLISHED NICKEL	24	24	
Brand#41	ECONOMY BRUSHED STEEL	9	24	
Brand#41	LARGE BRUSHED STEEL	48	24	
Brand#41	LARGE PLATED COPPER	36	24	
Brand#42	ECONOMY PLATED BRASS	3	24	
Brand#42	ECONOMY POLISHED TIN	6	24	
Brand#42	STANDARD ANODIZED NICKEL	12	24	
Brand#43	PROMO BRUSHED TIN	3	24	
Brand#43	SMALL ANODIZED COPPER	36	24	

+-----+-----+-----+-----+

only showing top 20 rows

Row count: 18232

```
[14]: # --- Decorrelated, deduped view for linear answering ---
q16_view = """
SELECT DISTINCT
  p.p_brand,
  p.p_type,
  p.p_size,
  ps.ps_suppkey
FROM part p
JOIN partsupp ps
  ON p.p_partkey = ps.ps_partkey
JOIN supplier s
  ON s.s_suppkey = ps.ps_suppkey
WHERE s.s_comment NOT LIKE '%Customer%Complaints%'
"""

spark.sql(f"CREATE OR REPLACE TEMP VIEW V_q16 AS {q16_view}")
q16_linear = f"""
SELECT
  p_brand, p_type, p_size,
  COUNT(*) AS supplier_cnt
FROM V_q16
WHERE p_brand <> '{BRAND}'
  AND p_type NOT LIKE '{TYPE_PREFIX}%'
  AND p_size IN ({sizes_csv})
GROUP BY p_brand, p_type, p_size
```

```

ORDER BY supplier_cnt DESC, p_brand, p_type, p_size
"""

print("Linearized Query 16 Results with view:")
q16_res_linear = spark.sql(q16_linear)
row_count = q16_res_linear.count()

q16_res_linear.show(truncate=False)

print(f"Row count: {row_count}")

q16_res_linear.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("./output/no_dp/q16_result_csv")

from pyspark.testing import assertDataFrameEqual
try:
    assertDataFrameEqual(q16_res, q16_res_linear)
    print("The DataFrames are identical (schema and content match).")
except AssertionError as e:
    print("The DataFrames are different.")
    print(e)

```

Linearized Query 16 Results with view:

p_brand	p_type	p_size	supplier_cnt
Brand#25	PROMO BRUSHED STEEL	24	28
Brand#41	ECONOMY ANODIZED STEEL	48	28
Brand#41	MEDIUM BRUSHED TIN	3	28
Brand#11	STANDARD BURNISHED BRASS	36	24
Brand#14	PROMO PLATED COPPER	48	24
Brand#15	MEDIUM ANODIZED NICKEL	3	24
Brand#21	MEDIUM ANODIZED COPPER	3	24
Brand#22	SMALL BRUSHED NICKEL	3	24
Brand#22	STANDARD ANODIZED TIN	12	24
Brand#25	MEDIUM BURNISHED COPPER	36	24
Brand#31	PROMO POLISHED COPPER	36	24
Brand#35	PROMO POLISHED NICKEL	24	24
Brand#41	ECONOMY BRUSHED STEEL	9	24
Brand#41	LARGE BRUSHED STEEL	48	24
Brand#41	LARGE PLATED COPPER	36	24
Brand#42	ECONOMY PLATED BRASS	3	24
Brand#42	ECONOMY POLISHED TIN	6	24
Brand#42	STANDARD ANODIZED NICKEL	12	24
Brand#43	PROMO BRUSHED TIN	3	24
Brand#43	SMALL ANODIZED COPPER	36	24

```
+-----+-----+-----+-----+
only showing top 20 rows
Row count: 18232
The DataFrames are identical (schema and content match).
```